The Journal of Open Source Software

ETLForge: A unified framework for synthetic test-data generation and ETL validation

Kyriakos Kartas [1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Yuanqing Wang

Reviewers:

- @gwbischof
- @Nelly-Barret

Submitted: 18 June 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

ETLForge is a lightweight Python package (Python 3.9) that generates realistic synthetic test data *and* validates ETL outputs using the **same declarative schema**. One YAML/JSON file describes field types, ranges, uniqueness, nullability and optional Faker templates (Faker Contributors, 2024). The schema is consumed by two high-level components: `DataGenerator` (creates CSV/Excel datasets) and `DataValidator` (checks pipeline outputs, returning row-level error reports). A Click-based CLI (The Pallets Projects, 2023) mirrors the library API, enabling automation within CI/CD workflows (Fowler & Foemmel, 2013). ETLForge therefore removes duplicated specifications and closes gaps where data-quality regressions can slip through testing cycles.

Statement of need

Extract-Transform-Load (ETL) processes are critical for data-driven organizations, but testing these pipelines remains challenging (Kimball & Ross, 2013; Kleppmann, 2017). Current testing approaches suffer from a fundamental disconnect: synthetic test data generation and output validation require separate tool chains with independent schema definitions. This creates several documented problems:

1. **Schema drift**: When generation schemas diverge from validation rules, tests may pass while production data fails validation (Redman, 2016).
2. **Maintenance overhead**: Duplicate schema definitions require synchronized updates, increasing development time and error potential (Dasu & Johnson, 2003).
3. **Testing gaps**: Inconsistent test data may not exercise edge cases that production validation catches, leading to false confidence (Loshin, 2010).

Existing libraries focus on either data generation (e.g., *Faker* (Faker Contributors, 2024)) or validation (e.g., *Great Expectations* (The Great Expectations Team, 2023), *pandera* (The pandera development team, 2023)). Because these tools are independent, engineers must maintain **parallel schemas**—one for generation, one for validation—leading to drift and missed bugs. ETLForge unifies both stages under a single source of truth, reducing maintenance effort and improving test robustness. Its small dependency footprint (six runtime packages) fits comfortably inside continuous-integration pipelines.

State of the field

The landscape of data generation and validation tools shows clear specialization, with tools excelling at either generation or validation, but lacking integration between these two complementary tasks:

Capability	ETLForge	Faker	Great Expectations	pandera	Cerberus
Schema-driven generation	Yes	Manual scripting	No	No	No
Schema-driven validation	Yes	No	Yes	Yes	Yes
Single schema for both	Yes	No	No	No	No
CLI & Python API	Both	CLI only	Both	Python only	Python only
YAML/JSON schema support	Yes	No	Python/YAML	Python only	Python only
Lightweight dependencies	Yes (6 core)	Yes (1 core)	No (20+ deps)	Yes (5 core)	Yes (0 core)

This comparison highlights that while several mature tools exist for data validation (Great Expectations, pandera, Cerberus) and Faker provides excellent data generation capabilities, none integrate both functions under a single schema definition. ETLForge is the only tool in this comparison that supports schema-driven generation *and* validation using the same configuration file, eliminating the need to maintain parallel schema definitions. Great Expectations offers the most comprehensive validation features but requires substantially more dependencies, making it less suitable for lightweight CI/CD environments. Faker requires manual scripting to define generation patterns rather than declarative schemas. To our knowledge, no existing open-source project provides an integrated, schema-first workflow covering both generation and validation with a unified configuration format.

Software description

ETLForge implements a dual-purpose architecture where a single YAML/JSON schema drives both data generation and validation processes. The schema format supports common data types (integer, float, string, date, category), constraints (ranges, uniqueness, nullability) and realistic data generation via Faker integration.

Core components: - `DataGenerator`: Creates synthetic datasets using pandas (McKinney & others, 2010) and numpy (Harris et al., 2020) for numerical operations - `DataValidator`: Validates CSV/Excel files against schema rules, returning detailed error reports - CLI interface: Enables command-line automation via Click (The Pallets Projects, 2023)

Typical workflow: ETLForge is designed to support the following pipeline:

1. **Schema definition**: Define data structure and constraints based on target system requirements
2. **Test data generation**: Generate synthetic datasets for initial ETL pipeline development and unit testing
3. **Pipeline validation**: Use the same schema to validate production or external data after ETL transformations
4. **Quality assurance**: Identify discrepancies between expected schema constraints and actual data quality

This workflow demonstrates that while ETLForge generates synthetic test data, its primary value proposition is in the validation phase where real production data is checked against expected constraints. The generation capability primarily serves to create controlled test datasets for unit testing ETL transformations before production data becomes available.

Integration approach: Rather than replacing existing tools, ETLForge complements ETL testing workflows by ensuring test data and validation rules remain synchronized. It integrates with pandas-based pipelines and exports to common formats (CSV, Excel via openpyxl). The framework targets tabular data structures, which represent the majority of ETL use cases in relational database and data warehouse environments.

Software methodology

Data generation algorithm: The `DataGenerator` component parses the schema specification and creates pandas DataFrames by iterating through field definitions. For each field type, it applies the appropriate generation strategy:

- **Numeric fields** (int, float): Uses numpy's random number generation with specified ranges and precision constraints
- **String fields**: Generates random strings or invokes Faker methods when `faker_template` is specified
- **Date fields**: Samples dates uniformly within specified ranges using pandas datetime utilities
- **Category fields**: Samples from predefined value sets with uniform distribution
- **Uniqueness constraints**: Maintains sets of generated values to ensure uniqueness when required
- **Nullability**: Applies configurable null rates to nullable fields using random sampling

Validation algorithm: The `DataValidator` component performs multi-pass validation on input datasets:

1. **Schema conformance**: Verifies all required columns exist and no unexpected columns are present
2. **Type checking**: Validates each cell's data type matches schema specifications
3. **Constraint validation**: Checks range constraints, uniqueness requirements and categorical value memberships
4. **Null validation**: Ensures null values only appear in nullable fields
5. **Error aggregation**: Collects all validation failures with row and column identifiers for detailed reporting

The validation process short-circuits on structural errors (missing columns) but continues through all rows to provide comprehensive error reports. This design prioritizes complete feedback over early termination.

Quality control: GitHub Actions run comprehensive checks on Python 3.9-3.11, including unit tests achieving 77% line coverage across 587 statements, static analysis via flake8 and mypy and integration testing through end-to-end workflows. All checks complete in under 90 seconds on Ubuntu runners, supporting rapid development cycles.

Performance characteristics

ETLForge demonstrates suitable performance for CI/CD integration, generating 10,000 rows in approximately 6 seconds and validating 100,000 rows in approximately 2 seconds on standard hardware (Intel Core i7, 16GB RAM). These benchmarks were conducted using a representative schema containing 8 fields with varying complexity levels:

- 2 integer fields with range constraints (id: 1-10000000, age: 18-80)
- 1 float field with range constraints (30000.0-150000.0) and precision specifications
- 3 string fields, including two with Faker template integration (name, email) and one with length constraints
- 1 categorical field with 5 predefined values (Engineering, Marketing, Sales, HR and Finance)
- 1 date field with range constraints (2020-01-01 to 2024-12-31)

Performance scales approximately linearly with the number of rows and fields. Complex constraints such as uniqueness checking and Faker integration introduce additional overhead but remain within acceptable bounds for typical testing scenarios. The complete benchmark schema is available in the repository as `benchmark_schema.yaml` for reproducibility.

These performance characteristics make ETLForge suitable for integration into continuous integration pipelines where Great Expectations may be too heavyweight for simple validation tasks, though ETLForge does not compete with Great Expectations' advanced statistical capabilities.

Discussion

ETLForge addresses a specific gap in the ETL testing ecosystem by unifying data generation and validation under a single schema, but makes deliberate trade-offs compared to specialized tools. Unlike Great Expectations (The Great Expectations Team, 2023), ETLForge does not provide data profiling, drift detection, or advanced statistical validations such as distributional analysis. Unlike pandera (The pandera development team, 2023), it lacks integration with Python type checkers and advanced pandas DataFrame validation patterns. ETLForge prioritizes simplicity and schema consistency over advanced analytical features, making it particularly well-suited for teams that need synchronized test data generation and validation without the complexity of enterprise-grade data quality platforms.

The framework currently has several technical limitations that constrain its applicability:

- **Dataset size**: Large datasets exceeding one million rows may require memory optimization strategies, as the current implementation loads entire datasets into pandas DataFrames during validation.
- **Nested structures**: Complex nested data structures are not supported. This limitation exists because ETLForge specifically targets tabular data formats (CSV and Excel) which are inherently flat. While YAML and JSON schema languages syntactically support nested structures, ETLForge intentionally focuses on relational and tabular ETL workflows where nested structures are less common. Future versions could support nested structures through flattening strategies or by targeting alternative output formats such as JSON documents.
- **Statistical validation**: Advanced statistical validations (distribution testing, anomaly detection and correlation analysis) require integration with specialized tools. ETLForge provides constraint-based validation rather than statistical analysis.
- **Custom validation logic**: While the framework validates against schema-defined constraints, it does not currently support user-defined validation functions, limiting extensibility for domain-specific validation rules.

Availability

The ETLForge source code is available on GitHub at https://github.com/kkartas/ETLForge under the MIT license. The latest release (v1.0.3) can be installed from the Python Package Index using `pip install etl-forge`, with an optional `etl-forge[faker]` installation variant for enhanced data generation capabilities. Complete documentation is hosted at https://etl-forge.readthedocs.io/. The software supports Linux, macOS and Windows operating systems and is compatible with Python versions 3.9 through 3.11.

References

Dasu, T., & Johnson, T. (2003). *Exploratory data mining and data cleaning*. John Wiley & Sons.

Faker Contributors. (2024). *Faker: A python package that generates fake data for you* (Version 24.4.0). https://doi.org/10.5281/zenodo.3823398

Fowler, M., & Foemmel, M. (2013). Continuous integration. *Thought-Works*. https://martinfowler.com/articles/continuousIntegration.html

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). John wiley & sons.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Loshin, D. (2010). *The practitioner's guide to data quality improvement*. Morgan Kaufmann.

McKinney, W., & others. (2010). Data structures for statistical computing in python. *Proceedings of the 9th Python in Science Conference*, *445*, 51–56. https://doi.org/10.25080/Majora-92bf1922-00a

Redman, T. C. (2016). *Getting in front on data: Who does what*. Harvard Business Review Press.

The Great Expectations Team. (2023). *Great Expectations: Always know what to expect from your data* (Version 0.18.0). https://doi.org/10.5281/zenodo.14976223

The Pallets Projects. (2023). *Click: Command Line Interface Creation Kit* (Version 8.1.3). https://doi.org/10.5281/zenodo.5123732

The pandera development team. (2023). *pandera: Data validation for scientists, engineers, and analysts* (Version 0.17.0). https://doi.org/10.25080/gerudo-f2bc6f59-010

DRAFT